UT3 · Subida segura de archivos

Recepción, validación y portadas de libros

Desarrollo web en entorno servidor · 2.º DAW

Juan Tamboleo López-Brea
Departamento de Informática
IES Ramón Arcas Meca · Lorca
Curso 2026/2027

Índice de la presentación

03–13
Recibir y diagnosticar
Temporal, estructura, errores y límites.

14–22
Validar y almacenar
MIME, nombre, destino y movimiento.

23–34
Integrar y editar
Publicación, sustitución y variante múltiple.

Flujo completo

RECIBIR Y DIAGNOSTICAR

1 · Recibir
multipart → $_POST y $_FILES
portada.jpg → temporal PHP
tmp_name señala el temporal

2 · Validar sin mover
Forma, error y tamaño
MIME mediante finfo
name/type/accept no bastan

3 · Comprobar operación
Campos del libro válidos
Cualquier error: no mover
El controlador recibe el resultado

Sin errores

4 · Guardar archivo
Nombre aleatorio y extensión
move_uploaded_file
storage/portadas

5 · Actualizar libro
Guardar solo su nombre
Fallo: retirar archivo nuevo
Éxito: retirar la antigua*

6 · Mostrar imagen
URL de img → otro GET
portada.php valida el nombre
readfile envía los bytes

*Tras el éxito y si no está compartida. El temporal sin mover se elimina al terminar la petición.

Formulario multipart

RECIBIR Y DIAGNOSTICAR

```
<form method="post" action="/libro-crear.php"
      enctype="multipart/form-data">
    <input name="titulo" value="Dune">
    <input name="portada" type="file"
           accept="image/jpeg,image/png,image/webp">
    <button type="submit">Guardar</button>
</form>
```

Codificación

multipart/form-data permite enviar el archivo.

Datos

El texto llega a $_POST y el archivo a $_FILES.

accept

Orienta el selector del navegador. El servidor vuelve a comprobar.

Un archivo en $_FILES

Clave	Valor de ejemplo	Interpretación
name	portada.jpg	Nombre proporcionado por el cliente.
tmp_name	Ruta temporal del servidor	Archivo recibido por PHP.
error	`UPLOAD_ERR_OK`	Estado de la recepción.
size	182430	Tamaño expresado en bytes.
type	image/jpeg	MIME enviado por el cliente.

name y type son datos externos. No deciden el nombre final ni el formato aceptado.

Comprobar la forma de la entrada

RECIBIR Y DIAGNOSTICAR

```
$archivo = $_FILES['portada'] ?? null;
$estructuraValida = is_array($archivo)
    && isset($archivo['error'], $archivo['size'])
    && is_int($archivo['error'])
    && is_int($archivo['size'])
    && isset($archivo['tmp_name'])
    && is_string($archivo['tmp_name']);

if ($archivo !== null && !$estructuraValida) {
    $errores['portada'] = 'Entrada no válida.';
}
```

Caso esperado

Un único archivo tiene error y size escalares.

Caso rechazado

portada[] genera arrays donde se esperaba un archivo.

Después

Solo se consultan los datos de subida si la estructura es válida.

Archivo opcional u obligatorio

Entrada	Portada opcional	Portada obligatoria
No hay archivo	Continuar sin portada.	Pedir que se seleccione.
`UPLOAD_ERR_NO_FILE`	Conservar la anterior al editar.	Mostrar error.
Archivo recibido	Validar todas sus propiedades.	Validar todas sus propiedades.
Entrada mal formada	Mostrar error.	Mostrar error.

En el ejemplo de biblioteca la portada es opcional.

Detener la validación ante un error

RECIBIR Y DIAGNOSTICAR

```
if ($archivo === null) {
    return ['error' => null, 'archivo' => null];
}
if (!$estructuraValida) {
    return ['error' => 'Entrada no válida.'];
}
if ($archivo['error'] === UPLOAD_ERR_NO_FILE) {
    return ['error' => null, 'archivo' => null];
}
if ($archivo['error'] !== UPLOAD_ERR_OK) {
    return ['error' => 'Falló la recepción.'];
}
```

Forma

La comprobación anterior calcula $estructuraValida.

Retorno

return sale de la función de validación, no de toda la página.

Resultado

El controlador recoge el error y puede mostrar de nuevo el formulario.

Errores durante la recepción

RECIBIR Y DIAGNOSTICAR

Código	Qué ha ocurrido	Respuesta
`UPLOAD_ERR_OK`	Recepción completada	Continuar con la validación.
`UPLOAD_ERR_INI_SIZE`	Supera el límite de PHP	Informar del límite.
`UPLOAD_ERR_PARTIAL`	Solo llegó una parte	Pedir un nuevo envío.
`UPLOAD_ERR_NO_TMP_DIR`	No hay directorio temporal	Revisar el servidor.
`UPLOAD_ERR_CANT_WRITE`	No se pudo escribir	Revisar almacenamiento.

No se trata un fallo de recepción como si fuera un archivo válido.

Otros códigos de recepción

Código	Significado	Consecuencia
`UPLOAD_ERR_FORM_SIZE`	Supera MAX_FILE_SIZE del formulario	Mostrar el error.
`UPLOAD_ERR_EXTENSION`	Una extensión de PHP detuvo la subida	Revisar el servidor.
`UPLOAD_ERR_NO_FILE`	No se seleccionó un archivo	Aplicar la regla opcional u obligatoria.

MAX_FILE_SIZE puede manipularse y no sustituye los límites del servidor.

Tamaño permitido y casos límite


```
$maxBytes = 2 * 1024 * 1024;
if ($archivo['size'] <= 0
    || $archivo['size'] > $maxBytes) {
    $errores['portada'] = 'Entre 1 byte y 2 MiB.';
}
```

Admitidos	Rechazados	Unidad
1 byte y 2 097 152 bytes pasan esta regla. Después todavía se comprueba el tipo.	0 y 2 097 153 bytes.	MiB expresa el límite de 1024 × 1024 bytes.

Límites antes del código PHP

Directiva	Ejemplo	Qué limita
upload_max_filesize	2M	Tamaño de cada archivo.
post_max_size	8M	Cuerpo completo del POST.
max_file_uploads	20	Número de archivos recibidos.
upload_tmp_dir	Directorio del servidor	Lugar donde PHP recibe temporales.

El cuerpo multipart incluye campos y separadores, además de los archivos.

Diagnosticar un envío demasiado grande

RECIBIR Y DIAGNOSTICAR

Síntoma	Posible causa	Comprobación
error = UPLOAD_ERR_INI_SIZE	Archivo mayor que upload_max_filesize	Comparar archivo y límite por archivo.
`$_POST y $_FILES vacíos`	POST mayor que post_max_size	Revisar el tamaño de la petición.
Faltan algunos archivos	Número mayor que max_file_uploads	Revisar cuántos se enviaron.
Fallo de escritura	Temporal o destino no escribible	Revisar permisos y espacio.

No confundir un formulario vacío con una recepción bloqueada por configuración.

Tres datos que pueden discrepar

VALIDAR Y ALMACENAR

Dato	Ejemplo	Quién lo aporta
Extensión del nombre	portada.jpg	Usuario.
MIME declarado	image/jpeg	Navegador o cliente.
MIME detectado	text/plain	finfo examina el contenido.

Un texto renombrado como .jpg se rechaza. Detectar el MIME es una comprobación, no una garantía absoluta de inocuidad.

finfo: examinar el contenido

VALIDAR Y ALMACENAR

```
new finfo(int $flags = FILEINFO_NONE, ?string $magic_database = null)
finfo::file(string $filename, int $flags = FILEINFO_NONE,
    $context = null): string|false
```

```
$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->file($archivo['tmp_name']);
```

Constructor

FILEINFO_MIME_TYPE pide el tipo MIME.

Método

file recibe una ruta de archivo del servidor.

Resultado

Una cadena como image/png o false si no puede obtenerla.

La extensión final la decide el servidor

VALIDAR Y ALMACENAR

```
$permitidos = [
    'image/jpeg' => 'jpg',
    'image/png' => 'png',
    'image/webp' => 'webp',
];
if ($mime === false || !isset($permitidos[$mime])) {
    $errores['portada'] = 'Formato no permitido.';
} else {
    $extension = $permitidos[$mime];
}
```

Caso correcto

image/png produce png.

Caso rechazado

text/plain no tiene una entrada permitida.

Orden

No acceder a $permitidos[$mime] antes de comprobarlo.

Archivos admitidos y rechazados

VALIDAR Y ALMACENAR

Archivo	Tamaño	MIME detectado	Decisión
foto.png	180 KiB	image/png	Admitir.
texto.jpg	2 KiB	text/plain	Rechazar tipo.
foto.webp	3 MiB	image/webp	Rechazar tamaño.
sin archivo	0	No se examina	Conservar portada.

El nombre original se puede mostrar escapado como información; nunca se usa directamente como destino.

Bytes aleatorios y texto hexadecimal

VALIDAR Y ALMACENAR

```
random_bytes(int $length): string
bin2hex(string $string): string
```

```php
$bytes = random_bytes(16);
$base = bin2hex($bytes);
```

random_bytes

Genera bytes aleatorios seguros. Puede lanzar una excepción.

bin2hex

Cada byte se representa con dos caracteres hexadecimales.

Resultado

16 bytes producen 32 caracteres hexadecimales.

Construir un nombre generado

VALIDAR Y ALMACENAR

```
try {
    $nombre = bin2hex(random_bytes(16))
        . '.' . $extension;
} catch (Random\RandomException) {
    $errores['portada'] = 'No se pudo preparar el archivo.';
}
```

Forma	Ejemplo de forma	Colisiones
32 caracteres hexadecimales y una extensión permitida.	a1…9f.png ilustra el formato; los puntos no forman parte del nombre real.	La aleatoriedad reduce mucho su probabilidad. El almacenamiento debe evitar sobrescribir nombres existentes.

Ruta física de destino

VALIDAR Y ALMACENAR

```
$directorio = __DIR__ . '/../storage/portadas';
$destino = $directorio . '/' . $nombre;
if (!is_dir($directorio) || !is_writable(
    $directorio)) {
    $errores['portada'] =
        'Almacenamiento no disponible.';
}
if (file_exists($destino)) {
    $errores['portada'] =
        'Nombre ocupado; vuelve a enviar.';
}
```

Preparación

El directorio se prepara en el servidor. No procede de un campo del formulario.

Destino

Se utiliza únicamente el nombre generado.

Concurrencia

La comprobación previa no es una reserva atómica del nombre.

move_uploaded_file

VALIDAR Y ALMACENAR

```
move_uploaded_file(string $from, string $to): bool
```

```
$guardado = move_uploaded_file(
    $archivo['tmp_name'], $destino
);
if (!$guardado) {
    $errores['portada'] =
        'No se pudo guardar la portada.';
}
```

Origen

Debe ser un archivo recibido por el mecanismo de subida HTTP de PHP.

Retorno

true al mover; false si no es una subida válida o falla el movimiento.

Advertencia

Si el destino existe, puede sobrescribirlo.

El movimiento también puede fallar

VALIDAR Y ALMACENAR

Situación	Resultado esperado	Tratamiento
Subida HTTP válida y destino correcto	true	Asociar el nombre al libro.
Archivo local creado desde CLI	false	No es una subida HTTP válida.
Directorio sin permisos	false y posible aviso	Informar sin publicar rutas internas.
Destino ya existente	Puede sobrescribirlo	Evitar ese destino.

La validación de tamaño y MIME debe haber terminado antes de mover.

Guardar no es publicar

INTEGRAR Y EDITAR

Ubicación	Acceso	Ejemplo
public/uploads	URL directa si el servidor la permite	/uploads/nombre.png
storage/portadas	Requiere una entrada PHP controlada	/portada.php?nombre=nombre.png
Ruta física	La utiliza PHP en el servidor	No se escribe como src de img.

El ejemplo principal guarda fuera de public. El nombre solicitado también se valida al servirlo.

Servir una portada: validar el nombre

INTEGRAR Y EDITAR

```
$nombre = $_GET['nombre'] ?? '';
if (!is_string($nombre) || preg_match(
    '/\A[a-f0-9]{32}\.(jpg|png|webp)\z/', $nombre
) !== 1) {
    http_response_code(404);
    exit;
}
$directorio = __DIR__ . '/../storage/portadas';
$ruta = $directorio . '/' . $nombre;
```

Entrada pública

El parámetro identifica solo un nombre generado.

Ruta fija

No se acepta ../, separadores ni una ruta completa.

Contexto

Primera parte de public/portada.php. Se completa en la página siguiente.

readfile: sintaxis y resultado

INTEGRAR Y EDITAR

```
readfile(string $filename, bool $use_include_path = false,
    $context = null): int|false
```

Parámetro	Retorno	Respuesta
filename indica la ruta física controlada.	Número de bytes enviados o false si falla.	La salida contiene los bytes del archivo. No se concatena su retorno al HTML.

Servir una portada: enviar el archivo

INTEGRAR Y EDITAR

```
$mime = is_file($ruta)
    ? (new finfo(FILEINFO_MIME_TYPE))->file($ruta) :
        false;
if (!in_array($mime, ['image/jpeg', 'image/png',
    'image/webp'], true)) {
    http_response_code(404);
    exit;
}
header('Content-Type: ' . $mime);
header('X-Content-Type-Options: nosniff');
readfile($ruta);
exit;
```

Continuación

La ruta ya tiene un nombre validado y un directorio fijo.

readfile

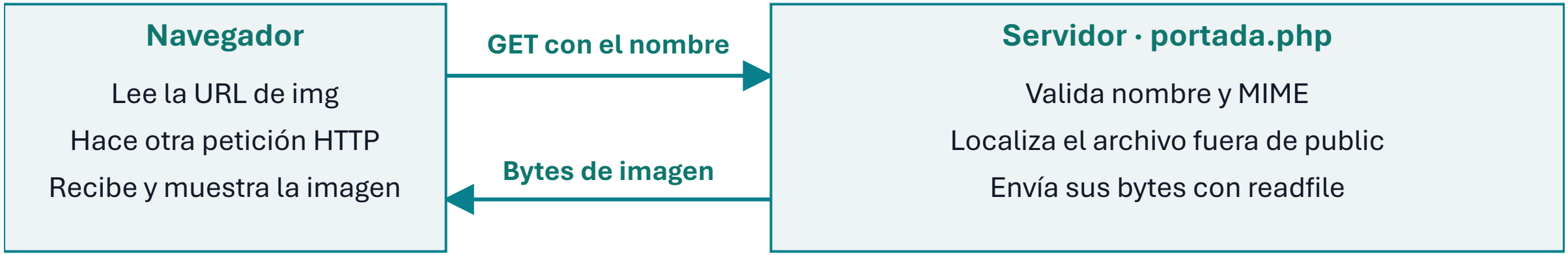
Escribe los bytes. Devuelve su cantidad o false.

Cabeceras

Content-Type declara el tipo. nosniff evita deducir otro.

La imagen utiliza una URL

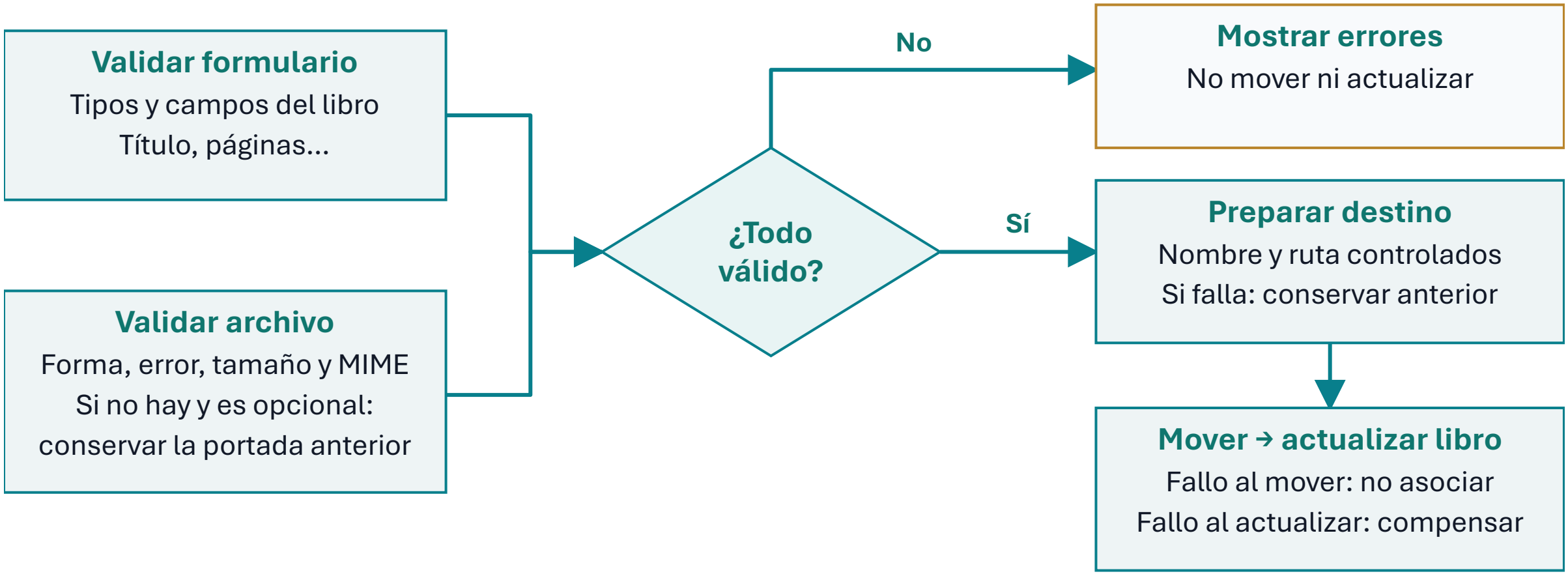
INTEGRAR Y EDITAR




```
<?php if ($libro->portada() !== null): ?>
    <img
        src="/portada.php?nombre=<?= e($libro->portada()) ?>"
        alt="Portada del libro">
<?php endif; ?>
```

Antes de mover: validar toda la operación

INTEGRAR Y EDITAR



Páginas inválidas + PNG válido → no se mueve el PNG. Un return devuelve el resultado al controlador.

Guardar solo una operación válida

INTEGRAR Y EDITAR

```
if ($errores === []) {
    // $libro ya se construyó con datos válidos.
    // Una portada presente ya se movió
    // correctamente.
    if ($nombrePortada !== null) {
        $libro->cambiarPortada($nombrePortada);
    }
    $id = $repo->guardar($libro);
    $operacionCorrecta = true;
}
```

Condición global

Cualquier error del formulario o del archivo impide guardar.

Sin portada

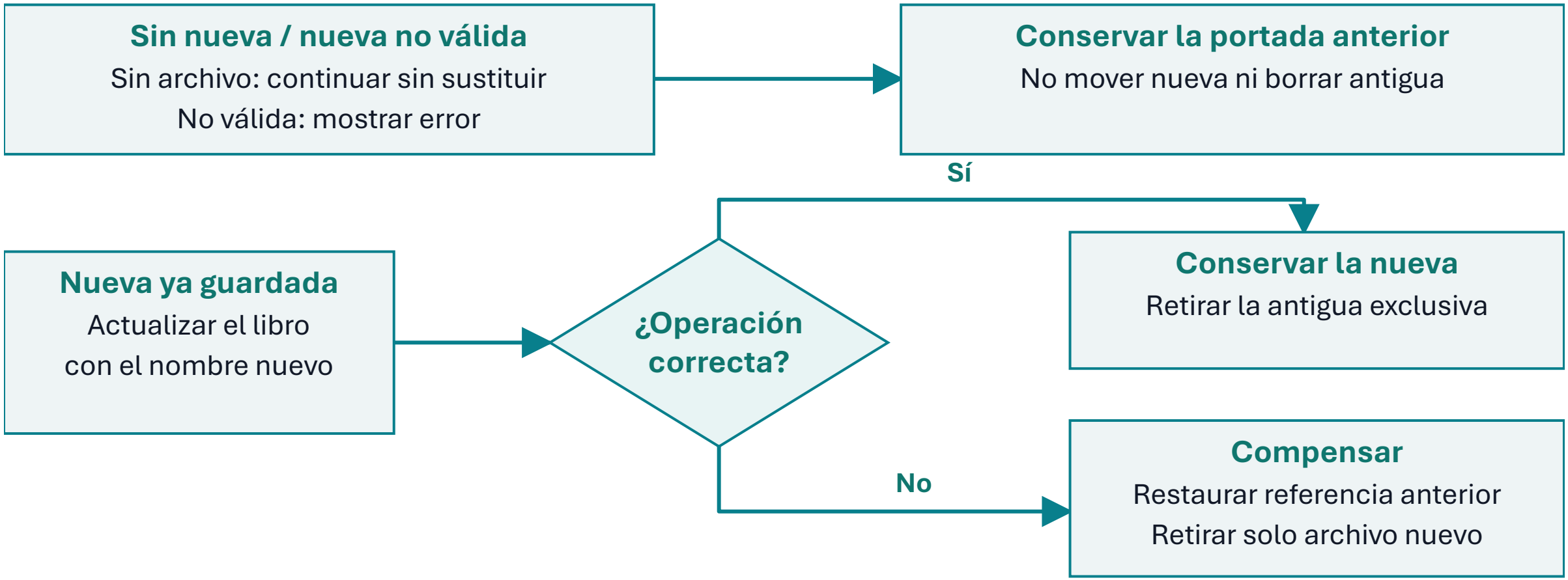
Un alta válida sigue guardándose.

Dos ciclos de vida

El archivo persiste en disco; el repositorio en memoria se reconstruye en otra petición.

Conservar, sustituir y compensar

INTEGRAR Y EDITAR



Limpieza con nombres generados y rutas fijas. Un fallo al retirar la antigua se registra; no deshace el éxito.

Compensar si falla la actualización

INTEGRAR Y EDITAR

```
// $nuevo ya se movió; $rutaNueva es su ruta
// controlada.
$anterior = $libro->portada();
try {
    $libro->cambiarPortada($nuevo);
    $repo->actualizar($id, $libro);
} catch (Throwable $e) {
    $libro->cambiarPortada($anterior);
    if (is_file($rutaNueva) && !unlink($rutaNueva))
        {
        error_log(
            'No se pudo limpiar la portada nueva.');
    }
    throw $e;
}
```

Fallo de operación

Se restaura la referencia anterior y se intenta retirar solo el archivo nuevo.

Límite

El ejemplo distingue compensación y almacenamiento persistente; no promete una transacción de base de datos.

Retirar la antigua después del éxito

INTEGRAR Y EDITAR

```
unlink(string $filename, $context = null): bool
```

```
// Después del try/catch anterior, con actualización
// correcta.
if ($anterior !== null && $anterior !== $nuevo) {
    $rutaAnterior = $directorio . '/' . $anterior;
    if (is_file($rutaAnterior) &&
        !unlink($rutaAnterior)) {
        error_log(
            'No se pudo retirar la portada antigua.');
    }
}
```

Control

Nombre generado y ruta fija. La portada antigua no está compartida.

Retorno

unlink devuelve true al eliminar y false si falla.

Fallo de limpieza

Se registra sin deshacer la actualización correcta ni borrar la portada nueva.

Variante: varias portadas

INTEGRAR Y EDITAR

```
<input type="file" name="portadas[]" multiple>
<?php
foreach ($_FILES['portadas']['error'] as $i =>
    $error) {
    $tmp = $_FILES['portadas']['tmp_name'][$i];
    $size = $_FILES['portadas']['size'][$i];
    // Validar cada archivo y asociar su propio
    // error.
}
```

Estructura

PHP agrupa cada propiedad en un array de índices.

Antes del bucle

Comprobar forma, índices y número de archivos.

Política del ejemplo

Validar el conjunto completo antes de mover. Si falla un movimiento, retirar los nuevos de ese intento.

Recorridos de la portada

INTEGRAR Y EDITAR

Envío	Resultado del formulario	Resultado del archivo
Datos y PNG válidos	Operación correcta	Archivo nuevo y referencia.
Páginas inválidas y PNG válido	Error de páginas	No se mueve el PNG.
Datos válidos y texto.jpg	Error de portada	Se conserva la anterior.
Edición sin archivo	Cambios de datos	Se conserva la anterior.

Si hay un error, el navegador obliga a seleccionar otra vez el archivo; el resto de campos puede conservarse.